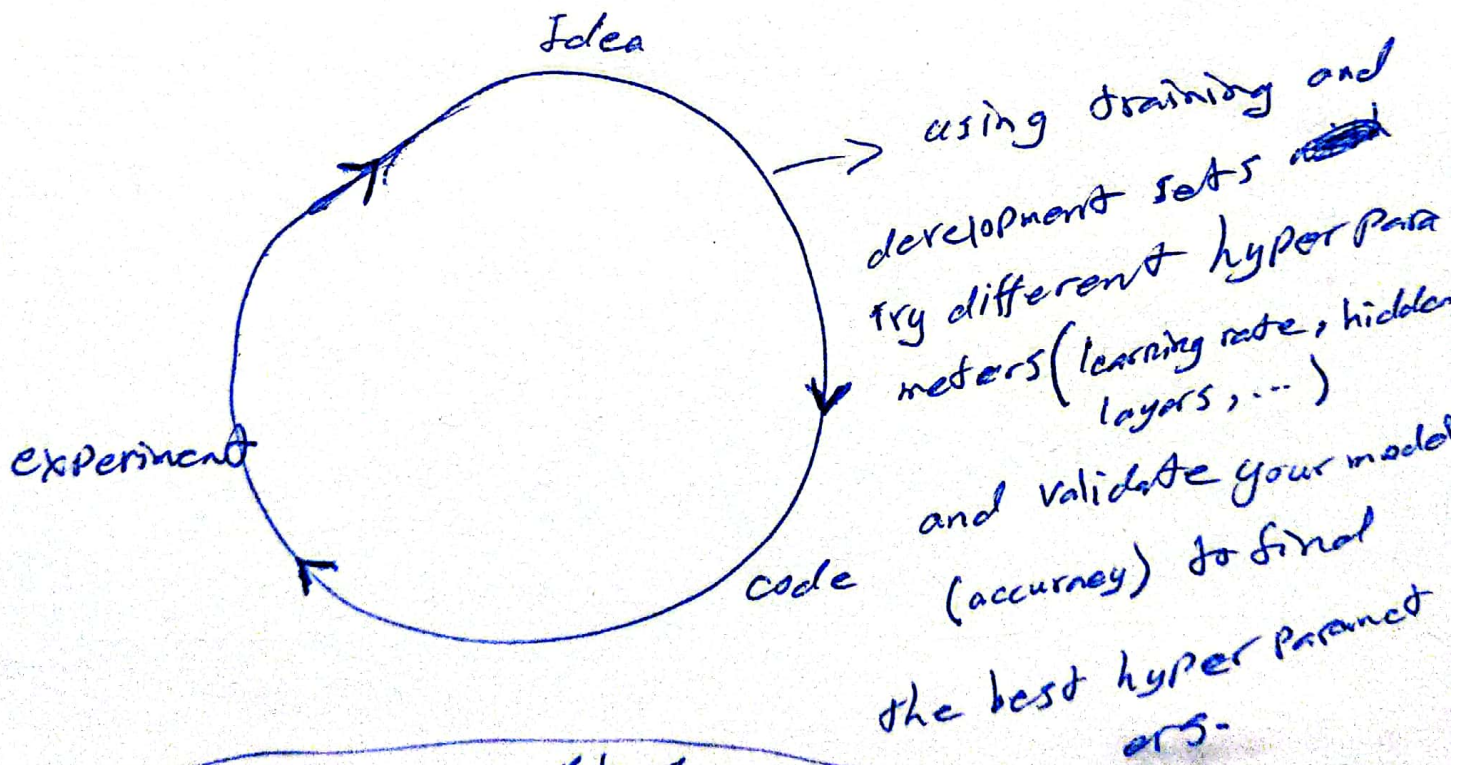
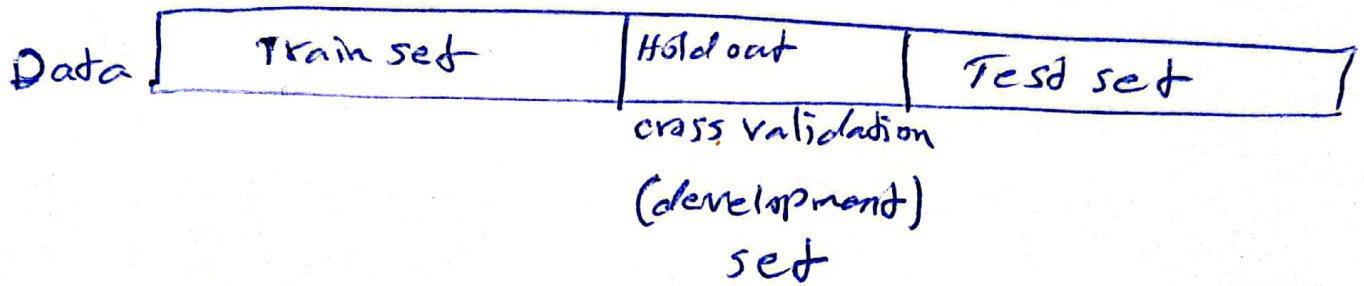


"Improving Deep Neural Networks"

⑦

(development)
Train/dev/test sets.



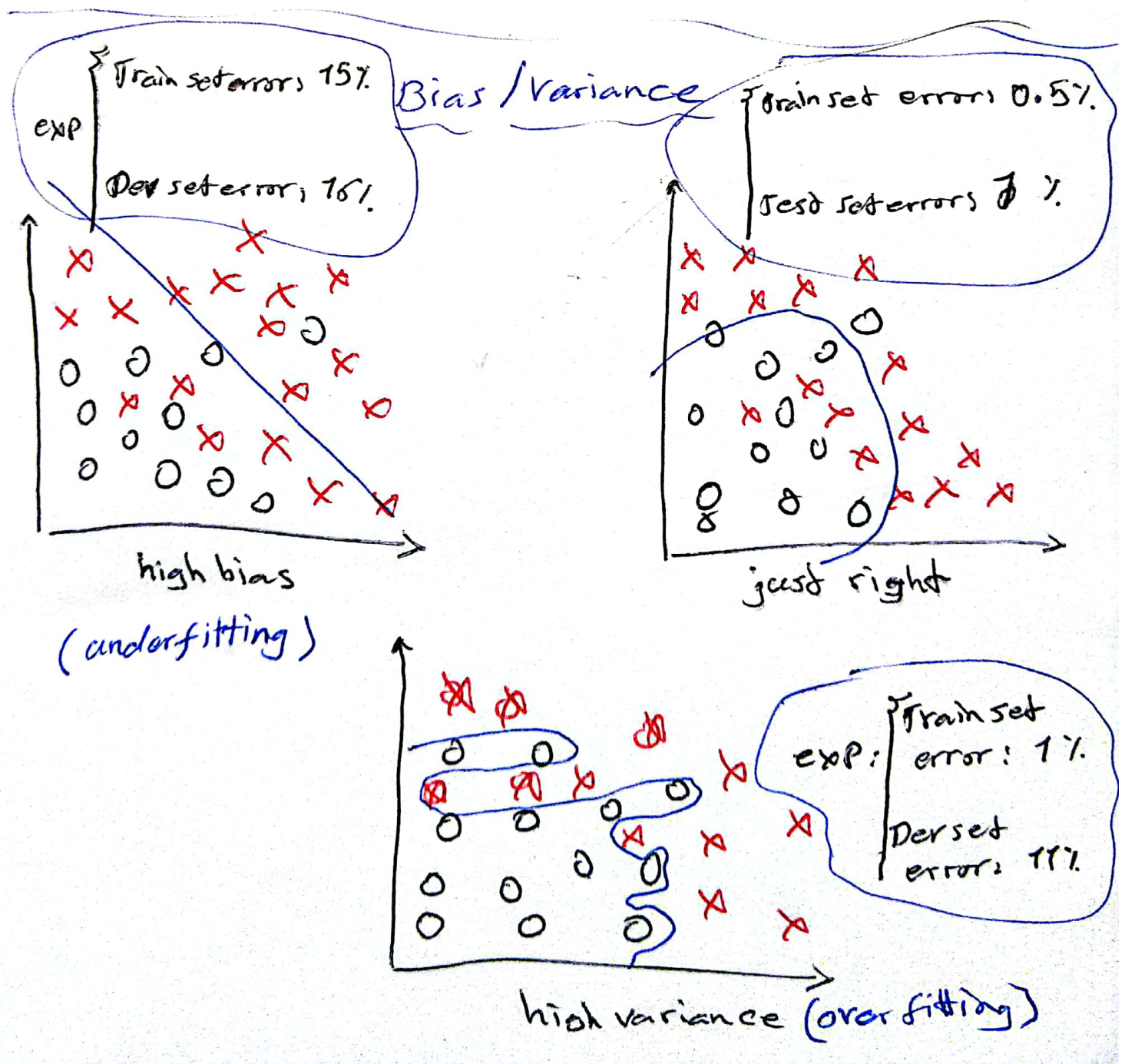
Big data : exp { data: 1000000
Train: 998000 (1.98)
dev: 10000 (1.1)
Test: 10000 (1.1)

mismatched Train/test distribution

(2)

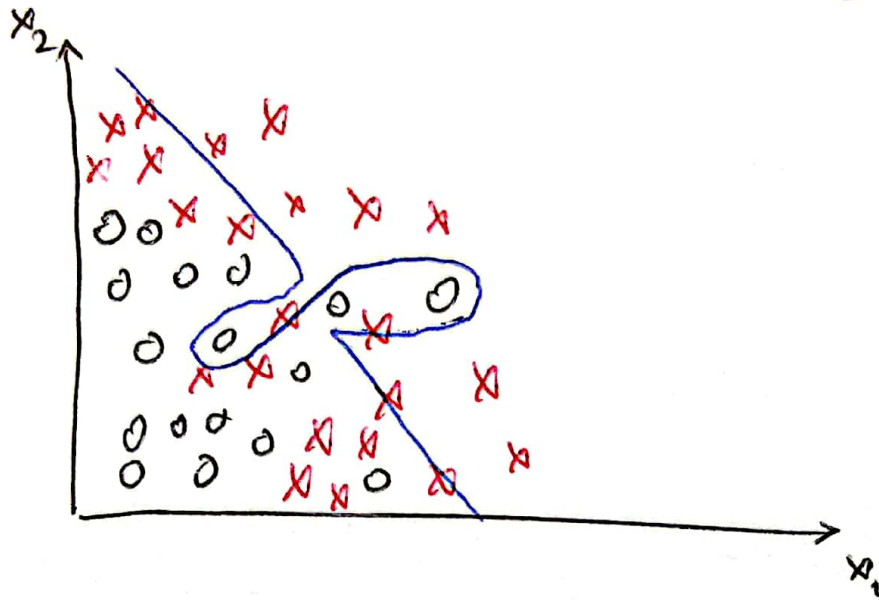
- make sure that dev and Test sets come from the same distribution.

- not having a test set might be okay. (only dev set.).

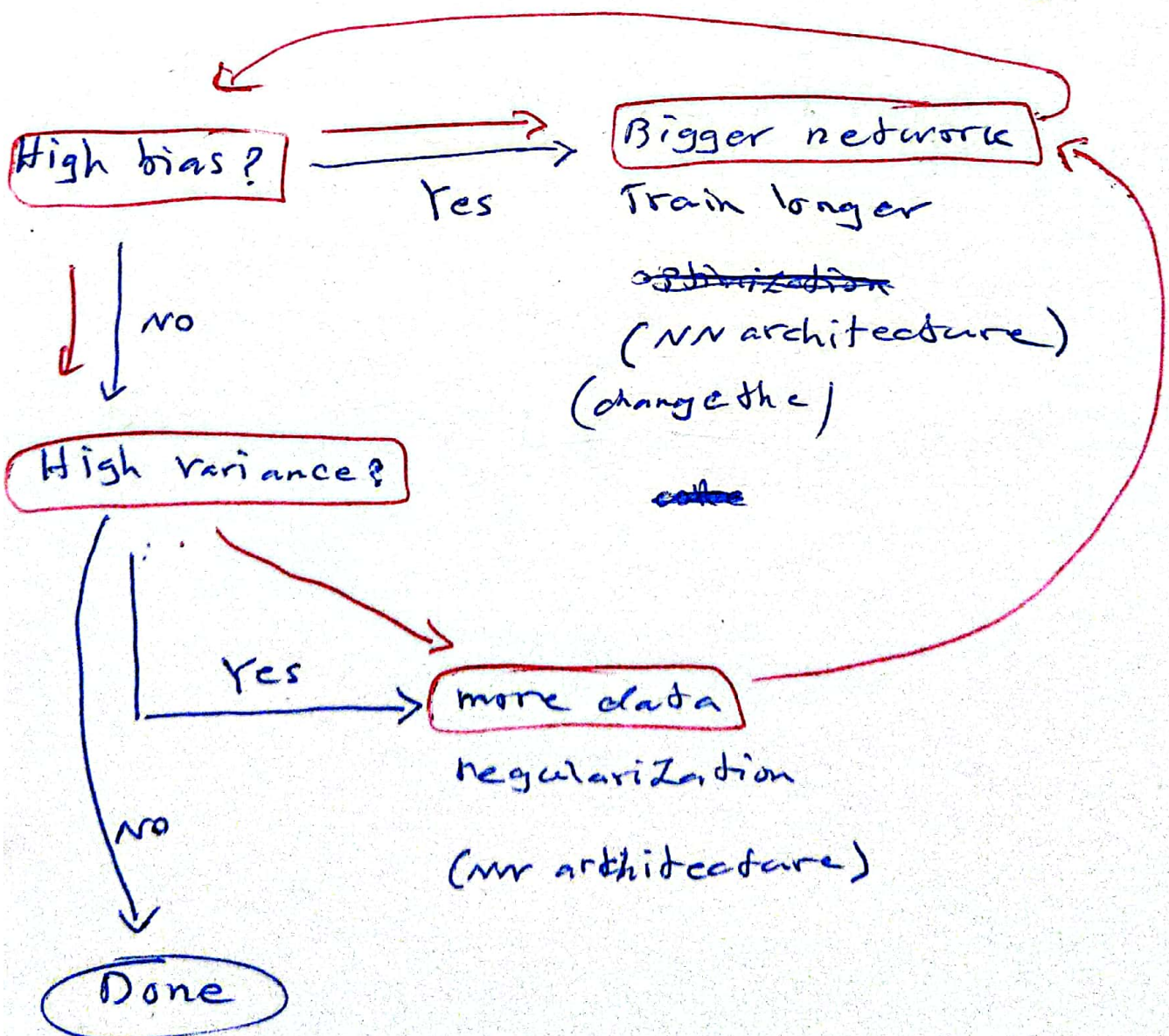


"high bias and high variance"

③



Basic recipe for machine learning



"Regularization"

(4)

logistic regression

$$\min_{(w,b)} J(w,b) \quad w \in \mathbb{R}^{n_x}, b \in \mathbb{R}$$

regularization
parameter

$$J(w,b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

L2 regularization: $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$
norm of (w^2)

L1 regularization $\frac{1}{m} \sum_{i=1}^{n_x} |w| = \frac{\lambda}{2m} \|w\|_1$

neural network

$$J(w^{(1)}, b^{(1)}, \dots, w^{(L)}, b^{(L)}) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

$$+ \frac{\lambda}{2m} \sum_{l=1}^L \|w^{(l)}\|^2$$

(5)

$$\|w^{[L]}\|_F^2 = \sum_{i=1}^n \sum_{j=1}^{n^{[L-1]}} (w_{ij}^{[L]})^2$$

$$w(n^{[L]}, n^{[L-1]})$$

"forbenius norm formula"

$$dw^{[L]} = (\text{back propagation}) + \frac{\lambda}{m} w^{[L]}$$

$$w^{[L]} := w^{[L]} - \alpha dw^{[L]}$$

$$\frac{dj}{dw^{[L]}} = dw^{[L]}$$

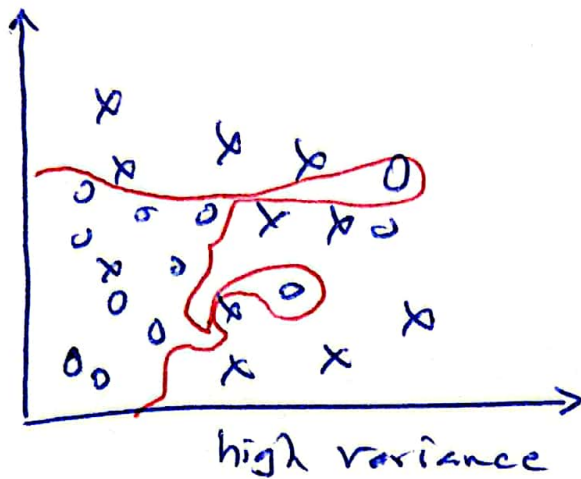
$$w^{[L]} := w^{[L]} - \alpha \left[w^{[L]} (\text{from backprop}) + \frac{\lambda}{m} w^{[L]} \right]$$

regularization sometimes called "weight decay"

$$w^{[L]} := w^{[L]} - \frac{\alpha \lambda}{m} w^{[L]} - \alpha (\text{from backprop})$$

$$= (1 - \frac{\alpha \lambda}{m}) w^{[L]}$$

How does regularization prevent overfitting? ⑥



$$J(w^{(L)}, b^{(L)}) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{(l)}\|_F^2$$

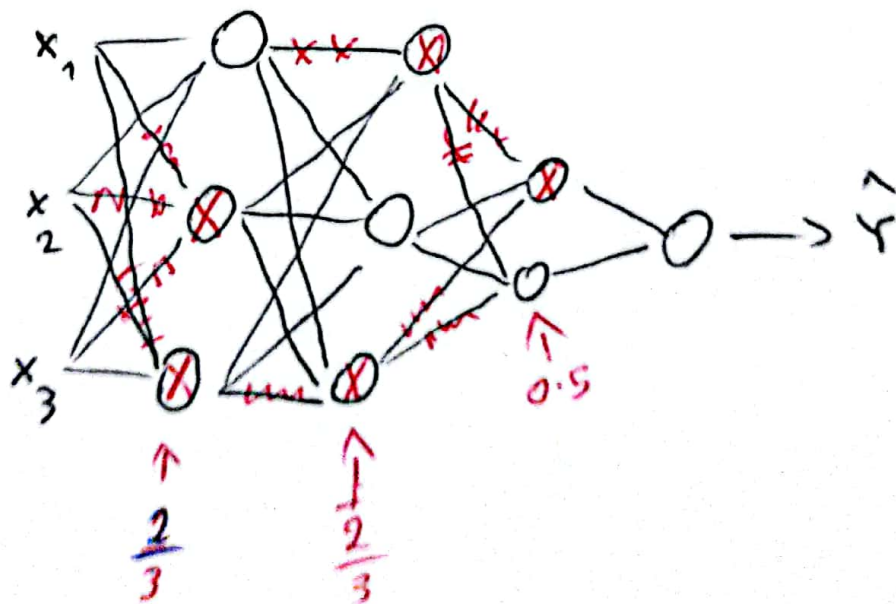
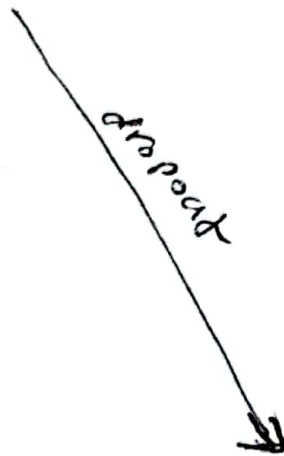
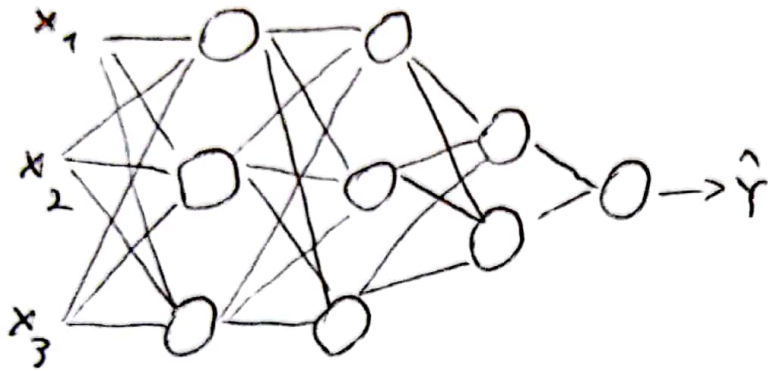
if lambda is a big number high variance $\rightarrow$ high bias

- A high lambda value increases the amount of regularization and creates ~~a higher~~ a simpler model with a higher bias but lower variance.
- A lower lambda decreases regularization, leading to a more complex model with lower bias but potentially higher ~~bias~~ variance.

Dropout Regularization

⑦

Practice of disregarding certain nodes in a layer at random during training, it ~~reduces~~ prevents overfitting



Implementing Dropout (invert dropout)

8

Illustrate with $L=3$

$$d3 = \text{np.random.rand}(a3.\text{shape}[0], a3.\text{shape}[1]) < \text{keep_prop}$$

$$a3 = \text{np.multiply}(a3, d3)$$

$$a3 /= \text{keep_prop}(0.8)$$

$\text{exp } 10.8$

eliminating 20% of
units in layer 3 ($d3$)

making predictions at test time

$$a^{[0]} = X$$

No dropout

$$z^{[1]} = w^{[1]} \cdot a^{[0]} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

$$z^{[2]} = w^{[2]} \cdot a^{[1]} + b^{[2]}$$

$$a^{[3]} = \dots$$

$$\dots \rightarrow \hat{y}$$

Note: The number of neurons in the previous layer gives us the number of columns of the weight matrix, and the number of neurons in the current layer gives us the number of rows in the weight matrix.

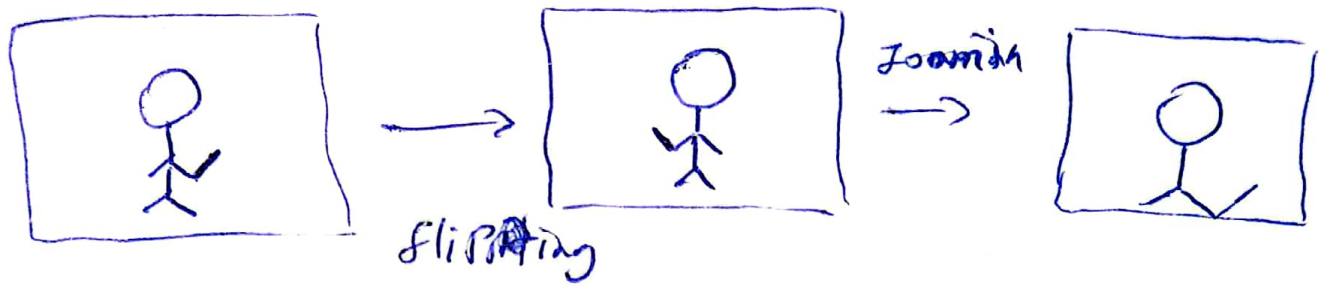
"understanding drop out"

- Intuition: Can't rely on any one feature, so have to spread out weights $\longrightarrow$ shrink weights
- dropout has the same effect as L2 regularization.
- The keep-prob (percentage of perceptrons which will be saved during dropout process in each layer) can be different for each layer.

Downside of dropout regularization ~~technique~~ technique is that the cost function (J) is not any more well defined.

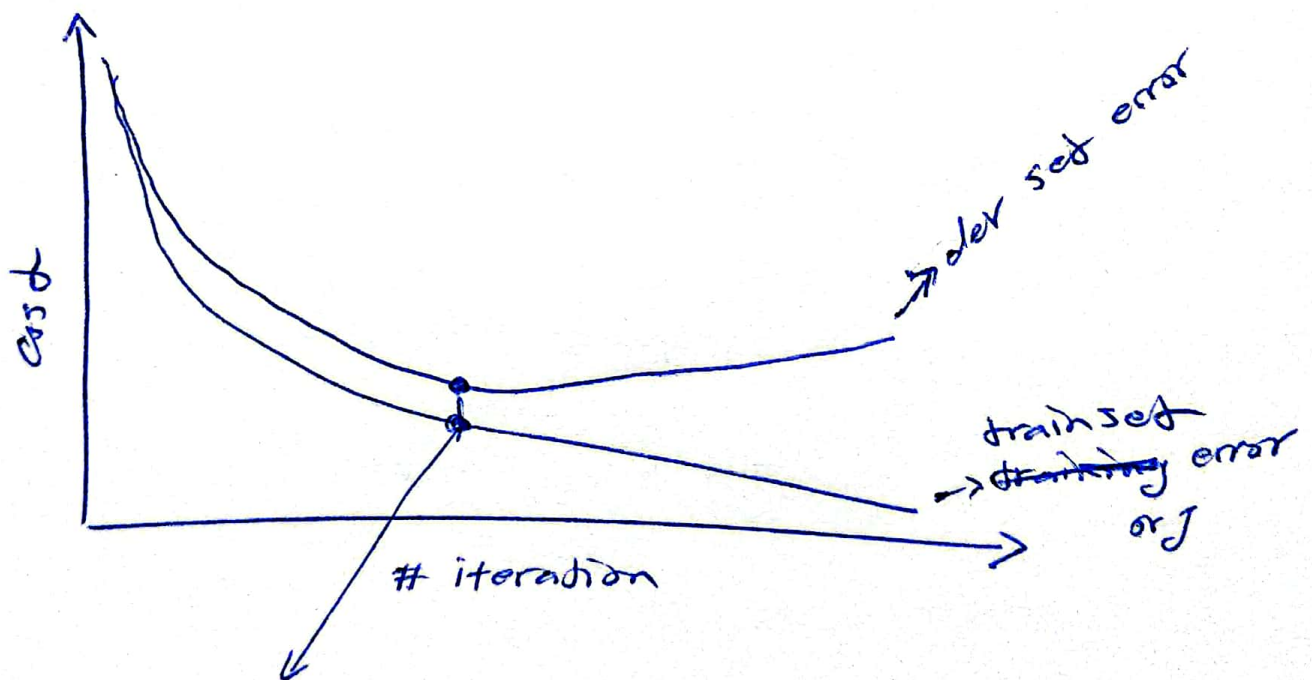
Data augmentation

10



rotate
~~rotation~~ the initial image

early stopping



downside of early stopping: by using early stop technique we stop the process of optimization of parameters by reducing cost ~~function~~ (J).

Regularization Techniques to reduce overfitting

(11)

1. L2-regularization
2. Dropout ~~technique~~ regularization technique.
3. Data augmentation
4. early stopping technique

Normalization

Normalizing training sets

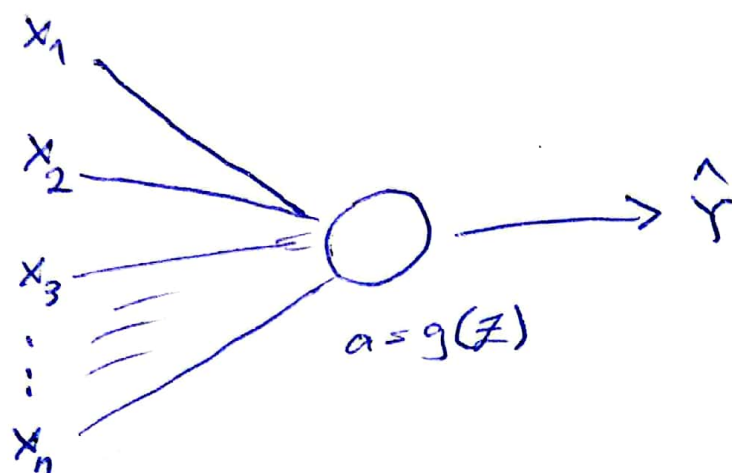
Standardizing :

$$Z = \frac{\vec{X} - \vec{\mu}}{\sigma}$$

vanishing / exploding Gradients

- The vanishing and exploding gradient problems occur when the gradients become either too small or too large during backpropagation. These occur in deep neural networks and one way to solve it is through initializing the parameters.

weight initialization for deep networks (12)



$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

large $n \longrightarrow$ small w

n = number of features

~~for~~ Variance(w) = $\frac{1}{n}$ in the case of ^{of} relu activation function.

$$w^{[L]} = \text{np.random.randn}(\text{shape}) \times \text{np.sqrt}\left(\frac{2}{L-1+n}\right)$$

in the case of Tanh activation function.

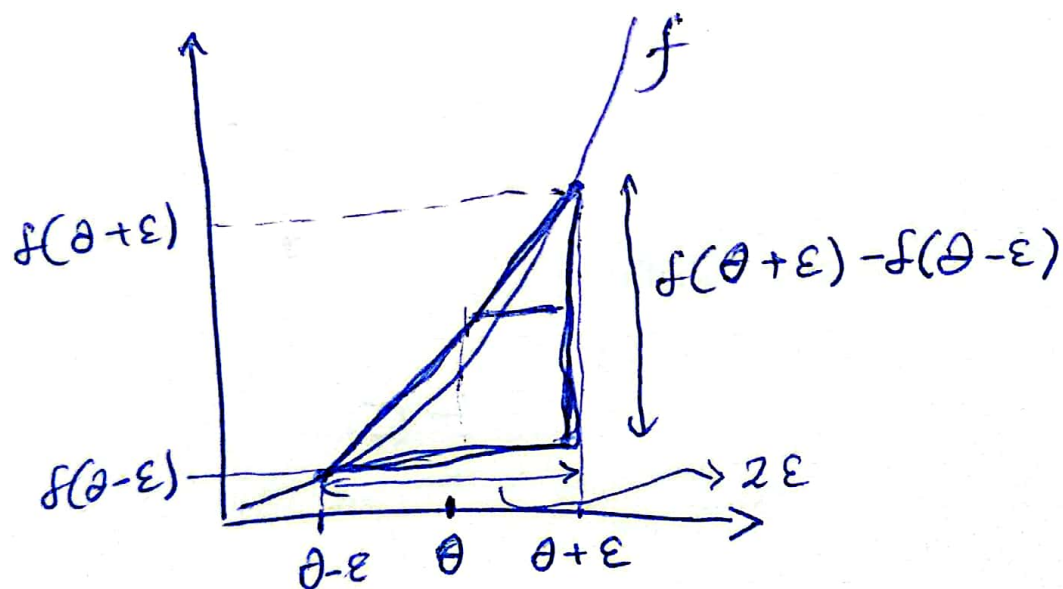
Xavier initialization

$$w^{[L]} = \text{np.random.randn}(\text{shape}) \times \text{np.sqrt}\left(\frac{1}{n(L-1)}\right)$$

$$W = \text{np.random.randn(shape)} * \text{np.sqrt}\left(\frac{2}{n^{[1]} + n^{[2]}}\right)$$

Numerical Approximation of Gradients

checking your derivative computation,



$$\epsilon = 0.01$$

$$\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon} \simeq g(\theta)$$

Gradient check for a neural network

1. Take $W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}$ and reshape into a big vector θ .

So,

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = J(\theta)$$

2. Take $dw^{[1]}, db^{[1]}, \dots, dw^{[L]}, db^{[L]}$ and reshape into a big vector $d\theta$.

Gradient checking (Grad check) $J(\theta) = J(\theta_1, \theta_2, \dots)$

for each i in θ :

$$d\theta_{\text{approx}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon}$$

↙

$$\approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$

check : $\frac{\|d\theta_{\text{approx}} - \partial\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|\partial\theta\|_2}$ $\approx$ ~~not~~ should be in the same range of ϵ

euclidean distance

- Do not use gradient checking in training - only to debug
- If algorithm fails grad check, look at components to try to identify bug.

components inside
of θ_{approx} and θ_1

- Remember regularization to calculate θ_{approx}
- Do not work with dropout.
- Run at random initialization; perhaps again after some training.

Optimization Algorithms

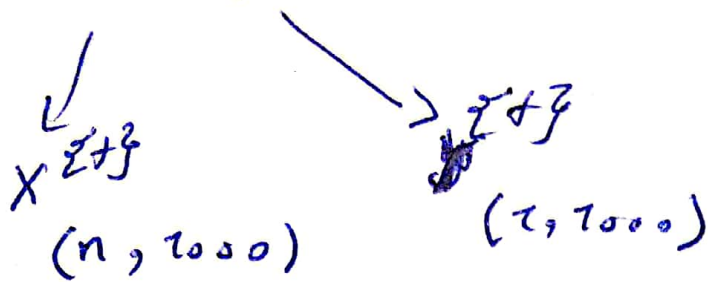
mini-batch gradient descent

$$X = \begin{bmatrix} X^{(1)} & X^{(2)} & \dots & X^{(1000)} & | & X^{(1001)} & \dots & X^{(2000)} & | & \dots & X^{(m)} \end{bmatrix}_{(n \times m)}$$

$X^{(1)} \dots X^{(1000)} \quad X^{(1001)} \dots X^{(2000)} \quad \dots \quad X^{(m)}$
 $X^{(1)} \dots X^{(1000)} \quad X^{(1001)} \dots X^{(2000)} \quad \dots \quad X^{(m)}$

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(1000)} & | & y^{(1001)} & \dots & y^{(2000)} & | & \dots & y^{(m)} \end{bmatrix}_{(1 \times m)}$$

each of $X^{[t]}, Y^{[t]}$ is a mini-batch training set (10)



mini-batch gradient descent, 5000 mini-batches in this exp.

for $t = 1, \dots, 5000$ in this example each mini-batch has 2000

1 step of gradient descent using $X^{[t]}, Y^{[t]}$

for prop on $X^{[t]}$:

$$Z^{[t]} = W^{[t]} \cdot X^{[t]} + b^{[t]}$$

$$A^{[t]} = g^{[t]}(Z^{[t]})$$

$$\text{cost function } J = \frac{1}{1000} \sum_{i=1}^{1000} \frac{1}{2} (\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2} \sum_{l=1}^L \|W^{[l]}\|_F^2$$

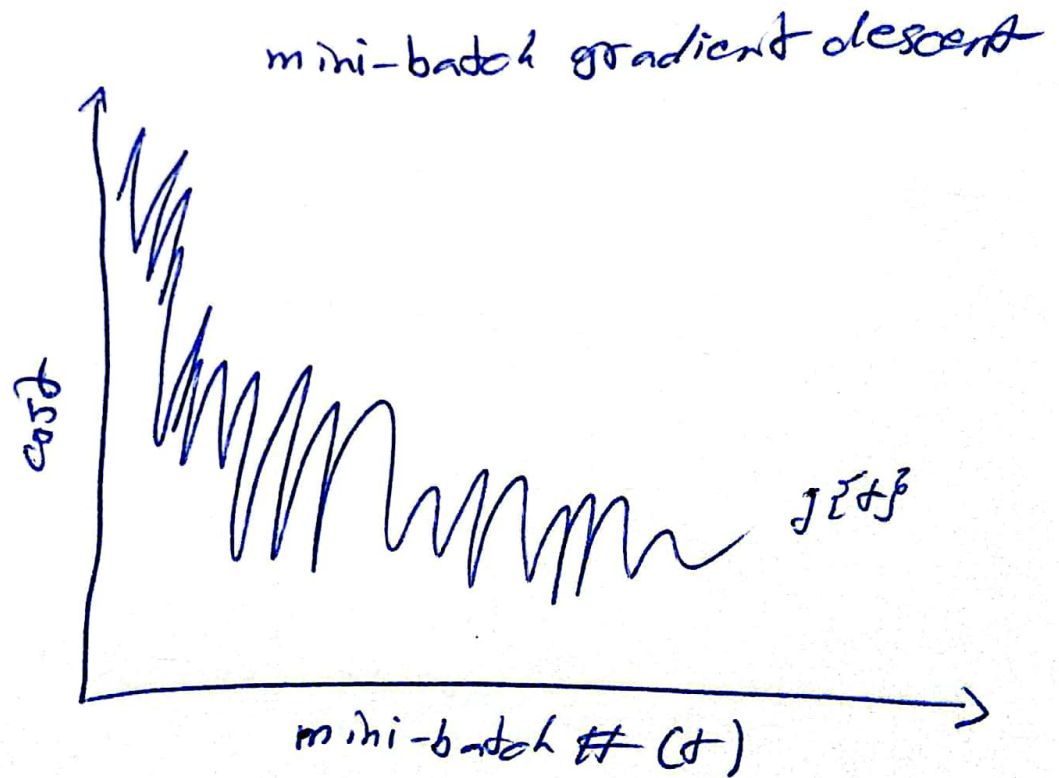
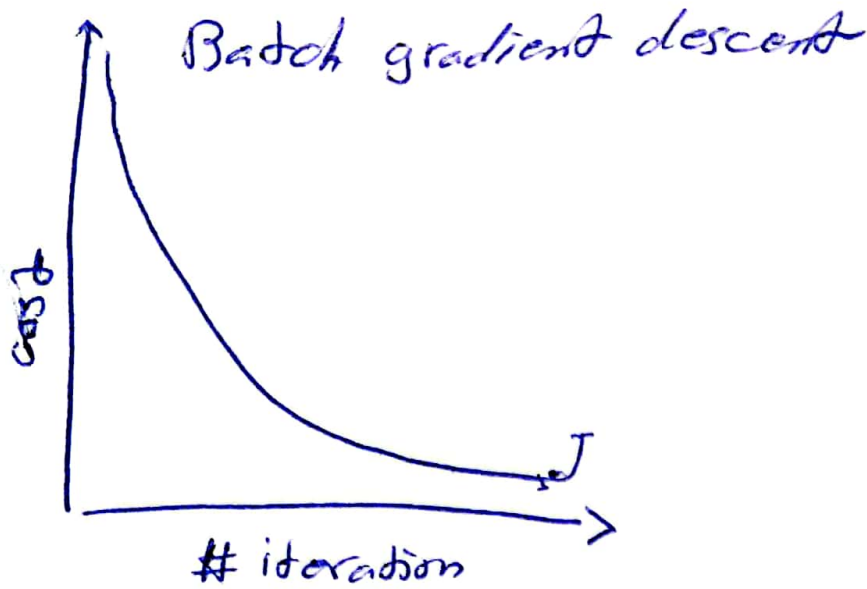
number of examples in mini-batches

regularization

back prop to compute gradients of $J^{[t]}$:

$$dW^{[l]} = W^{[l]} \times \alpha \frac{dJ}{dW^{[l]}}$$

$$db^{[l]} = b^{[l]} \times \alpha \frac{dJ}{db^{[l]}}$$



in each iteration we use a different training set (mini-batch) to train the model so the cost plot of cost function ~~is~~ is not the same as usual

- If mini-batch size = m so $\Rightarrow$ batch gradient descent.

- If mini-batch size = 1 so $\Rightarrow$ stochastic gradient descent

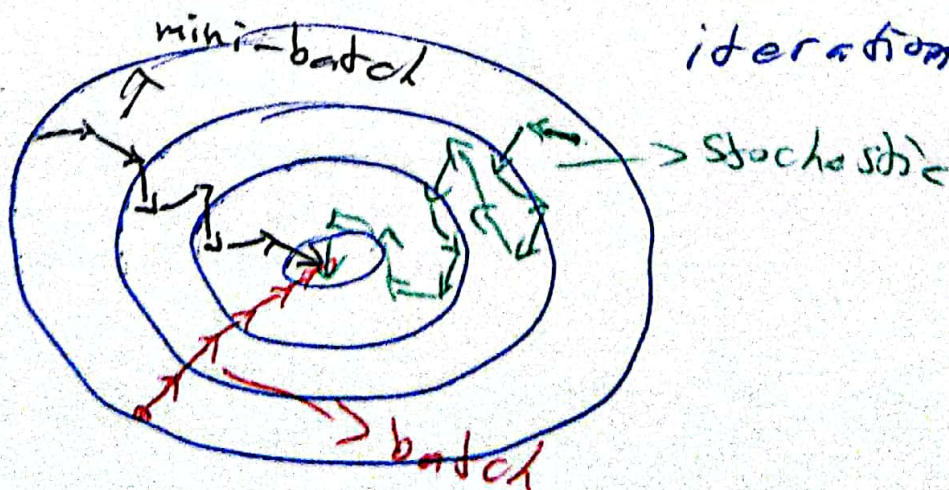
So,

batch gradient descent (uses all $\overbrace{\text{examples}}^m$ in each iteration)

~~mini-batch~~

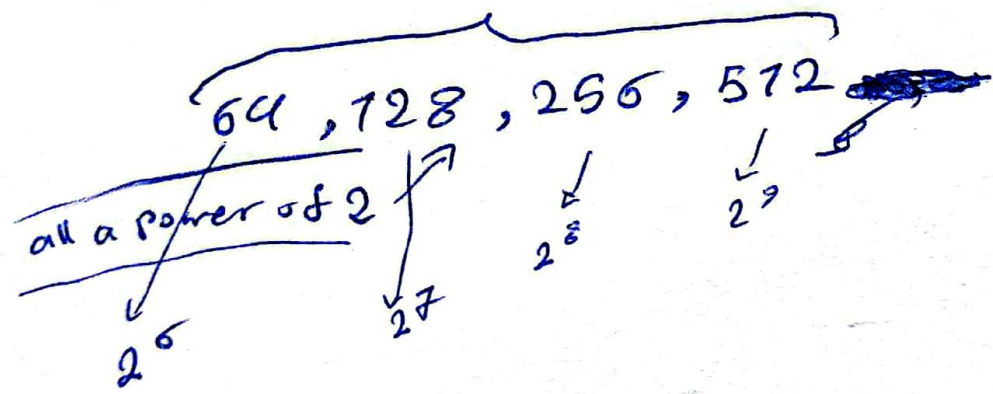
mini-batch gradient descent (uses a ^{diff} subset of m examples in each iteration)

stochastic gradient descent (uses just 1 example in each iteration)



- a downside of stochastic gradient descent is that we lose all the speed up ~~that~~ of vectorization.
- so the fastest learning for big dataset will be mini-batch gradient descent.

-
- if the training set is small $\rightarrow$ use batch gradient descent. (exp: less than 2000 examples)
 - Typical mini-batch sizes:



- mini-batch is a hyperparameter which should be optimized.

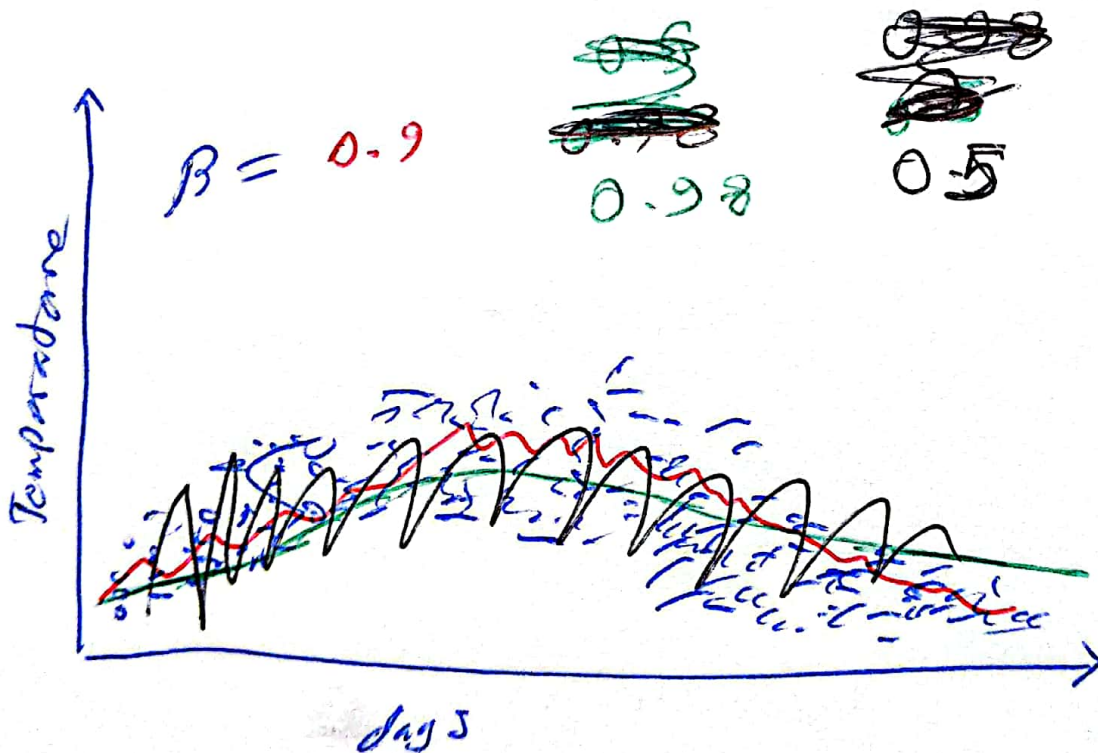
"Exponentially weighted averages"

(20)

$$V_t = \beta \cdot V_{t-1} + (1-\beta) \theta_t$$

exp:

$$V_{100} = 0.9 V_{99} + 0.1 \theta_{100}$$



$$V_{100} = 0.9 V_{99} + 0.1 \theta_{100}$$

$$V_{99} = 0.9 V_{98} + 0.1 \theta_{99}$$

$$V_{98} = 0.9 V_{97} + 0.1 \theta_{98}$$

if we expand V_{100} :

(27)

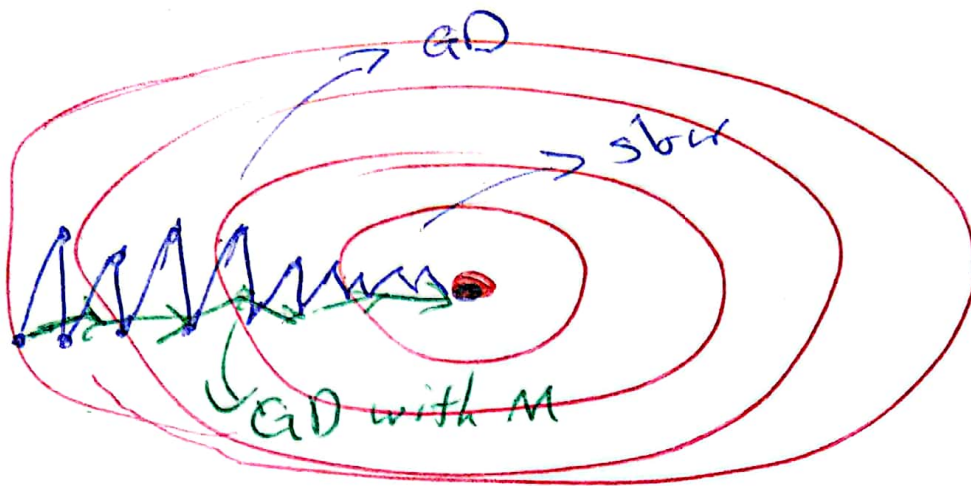
$$\begin{aligned}
 V_{100} &= 0.1 \theta_{100} + 0.9 V_{99} \\
 &\rightarrow (0.1 \theta_{99} + 0.9 \theta_{98}) \\
 &\rightarrow (0.1 \theta_{98} + 0.9 \theta_{97}) \\
 &= 0.1 \theta_{100} + 0.1 \times 0.9 \times \theta_{99} + 0.1 (0.9)^2 \theta_{98} + \dots
 \end{aligned}$$

Implementing exponentially weighted averages,

$$\left. \begin{aligned}
 V_0 &= 0 \\
 V_1 &= \beta V_0 + (1-\beta) \theta_1 \\
 V_2 &= \beta V_1 + (1-\beta) \theta_2 \\
 V_3 &= \beta V_2 + (1-\beta) \theta_3 \\
 &\dots
 \end{aligned} \right\} \begin{aligned}
 V_0 &= 0 \\
 V_1 &= \beta V_0 + (1-\beta) \theta_1 \\
 V_2 &= \beta V_1 + (1-\beta) \theta_2 \\
 &\vdots
 \end{aligned}$$

"Gradient Descent with momentum"

22



momentum:

on iteration t :

compute dw, db on current mini-batch
(or batch)

compute $v_{dw} = \beta \cdot v_{dw} + (1 - \beta) dw$



$$v_{\theta} = \beta v_{\theta} + (1 - \beta) \theta_t^u$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$w = w - \alpha v_{dw}$$

$$b = b - \alpha v_{db}$$

GD with Momentum Implementation details (23)

$v_{dw} = 0$; $v_{db} = 0$ $\rightarrow$ both matrix and vector with the same shape as dw and db initialize to 0.

On iteration t :
compute dw & db on the current mini-batch

$$\begin{aligned} v_{dw} &= \beta \cdot v_{dw} + (1 - \beta) dw \\ v_{db} &= \beta \cdot v_{db} + (1 - \beta) db \end{aligned} \quad \left. \begin{aligned} v_{dw} &= \beta \cdot v_{dw} + dw \\ v_{db} &= \beta \cdot v_{db} + db \end{aligned} \right\} \text{not intuitive}$$

$$\begin{aligned} w &= w - \alpha \cdot v_{dw} \\ b &= b - \alpha \cdot v_{db} \end{aligned} \quad \downarrow \text{preferred implementation}$$

hyperparameters: α & β

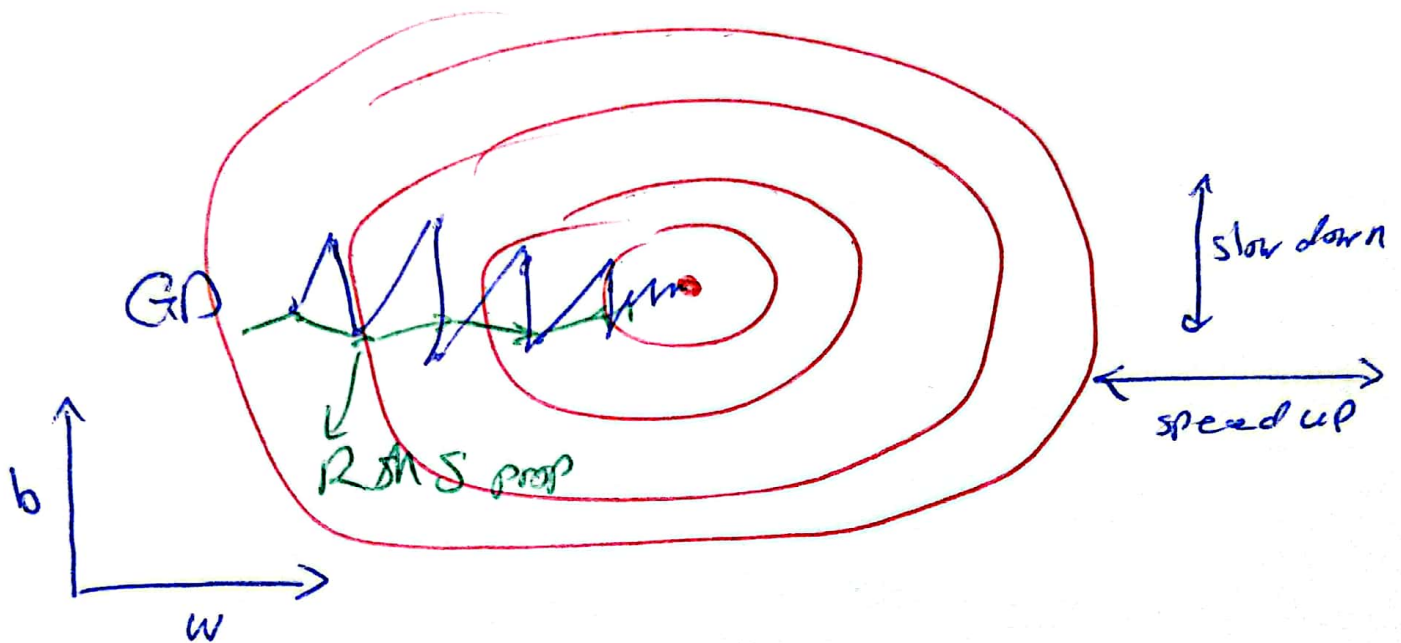
$\beta = 0.9$
 $\swarrow$
most common value

bias correction: $\left(\frac{v_{dw}}{1 - \beta^t} \right)$ and $\left(\frac{v_{db}}{1 - \beta^t} \right)$
 $\underbrace{\hspace{10em}}$
no need to do it

RMS prop

29

(Root Mean Square)



On iteration t :

compute dw , db on current mini-batch

$$s_{dw} = \beta \cdot s_{dw} + (1-\beta) \underline{dw}^2 \rightarrow \text{element wise square operation}$$

$\leftarrow$ relatively small value

$$s_{db} = \beta \cdot s_{db} + (1-\beta) \underline{db}^2 \leftarrow \text{relatively large value}$$

$$w := w - \alpha \frac{dw}{\sqrt{s_{dw}}} \rightarrow +\epsilon$$

$$b := b - \alpha \frac{db}{\sqrt{s_{db}}} \rightarrow +\epsilon$$

~~Zero division~~
Prevent Zero division

Adam optimization Algorithm

25

(Adaptive Moment Estimation)

Adam = RMSprop + momentum

$$v_{dw} = 0 ; s_{dw} = 0 ; v_{db} = 0 ; s_{db} = 0$$

On iteration t :

compute derivative dw, db using current mini-batch

$$v_{dw} = \beta_1 \cdot v_{dw} + (1 - \beta_1) dw \quad \text{"momentum" } \beta_1$$

$$v_{db} = \beta_1 \cdot v_{db} + (1 - \beta_1) db$$

$$s_{dw} = \beta_2 \cdot s_{dw} + (1 - \beta_2) dw^2 \quad \text{"RMSprop" } \beta_2$$

$$s_{db} = \beta_2 \cdot s_{db} + (1 - \beta_2) db^2$$

$$v_{dw}^{(corrected)} = v_{dw} / (1 - \beta_1^t)$$

$$v_{db}^{(corrected)} = v_{db} / (1 - \beta_1^t)$$

$$s_{dw}^{(corrected)} = s_{dw} / (1 - \beta_2^t)$$

$$s_{db}^{(corrected)} = s_{db} / (1 - \beta_2^t)$$

bias correction terms

$$w := w - \alpha \cdot \frac{Vdw^{(corrected)}}{\sqrt{Sdw^{(corrected)} + \epsilon}}$$

$$b := b - \alpha \cdot \frac{Vdb^{(corrected)}}{\sqrt{Sdb^{(corrected)} + \epsilon}}$$

HyperParameters of ADAM

α : needs to be tune

β_1 : 0.9 common value (dur)

β_2 : 0.999

ϵ : 10^{-8}

→ second moment
(dur²)

→ first moment

default

learning rate decay

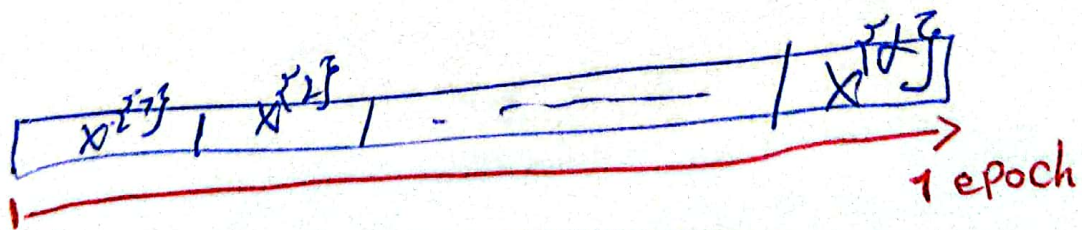
- reduce learning rate over time (iterations)

- why to implement learning rate decay?

exp:



1 epoch = 1 Pass through the data (training set)



$$\alpha = \frac{1}{1 + \underbrace{\text{decay rate} \times \text{epoch number}}_{\text{hyper parameter}}} \times \alpha_0$$

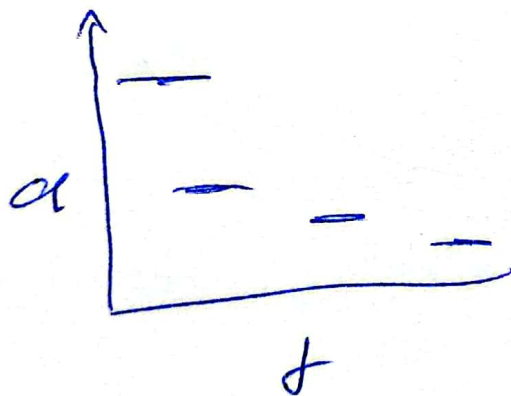
other learning rate decay methods

23

$$\alpha = 0.95^{\text{epoch-num}} \times \alpha_0$$

$$\alpha = \frac{k}{\sqrt{\text{epoch-num}}} \times \alpha_0$$

$$\alpha = \frac{k}{\sqrt{t}} \cdot \alpha_0$$



discrete stair case

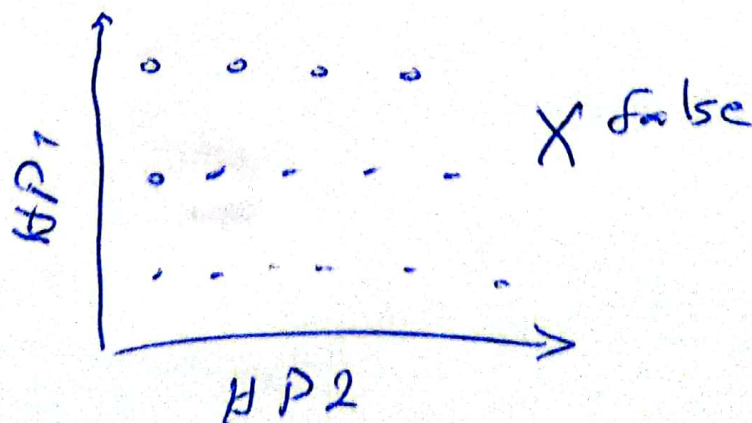
Tuning process

Hyperparameters

- α $\longrightarrow$ the most important HP to tune
- β $\longrightarrow$ momentum term : 0.9
- $\beta_1, \beta_2, \epsilon$ $\longrightarrow$ Adam Parameters
 - 0.9 $\longleftarrow \beta_1$
 - 0.999 $\longleftarrow \beta_2$
 - 10^{-8} $\longleftarrow \epsilon$
- # layers
- # hidden units
- learning rate decay
- mini-batch size

Notes:

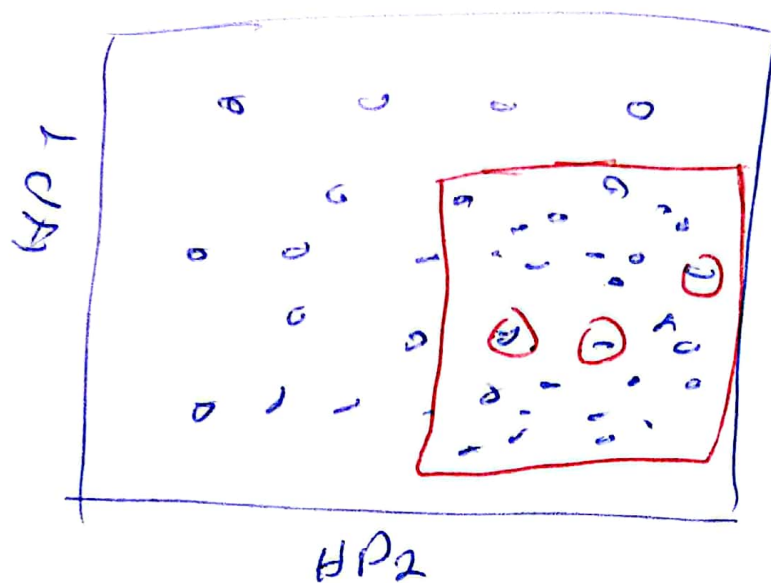
- Try random values : Do not use a grid



- choose the points randomly

- coarse to fine

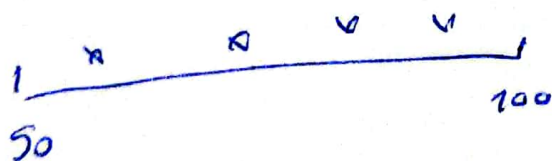
30



using

picking HPs at random

$$n^{[L]} = 50, \dots, 100$$



layers $L: 2 - 6$
2, 3, 4

$\alpha = ?$ $\rightarrow \frac{1 - \alpha}{10}$

$r = -4 \times \text{np.random.rand}()$

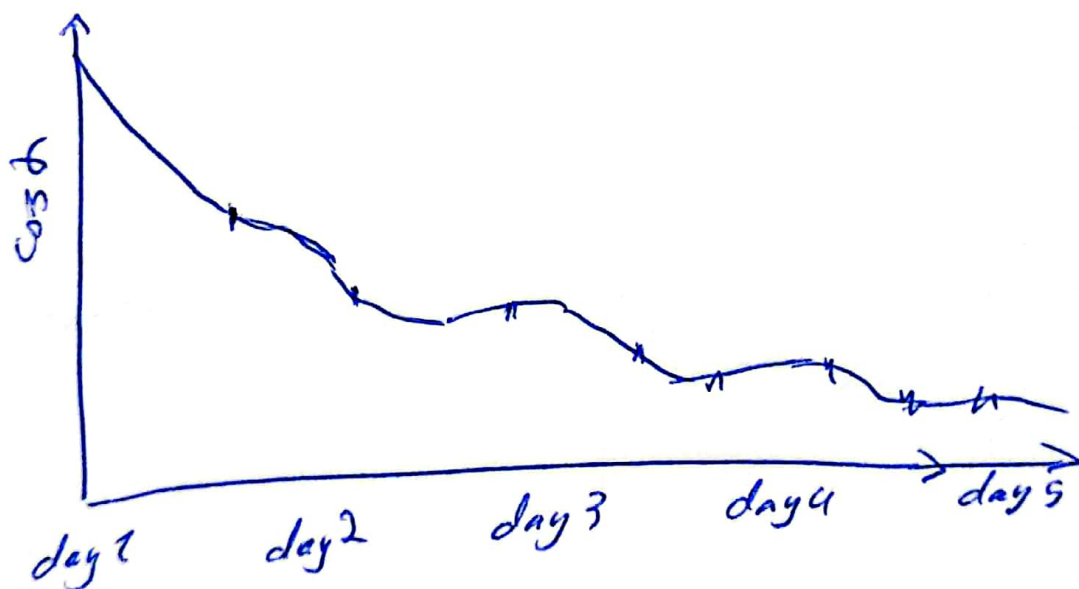
$\alpha = 10^r$

$$\underbrace{10^a \dots 10^b}$$

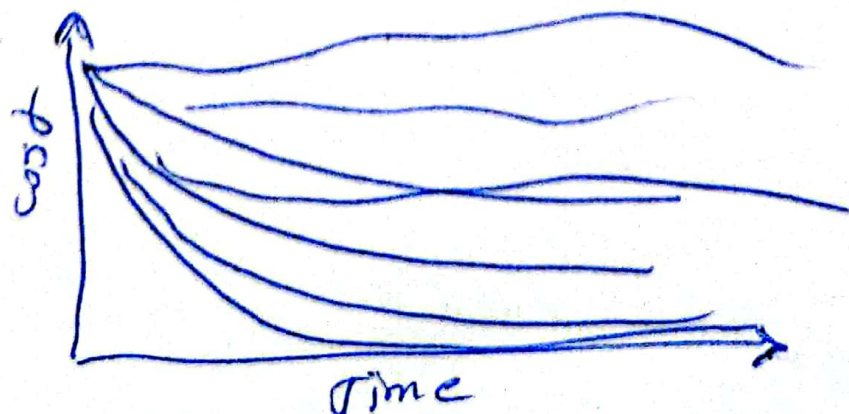
$$a = \log_{10} ?$$

$$b = \log_{10} ?$$

Babysitting one model ? (Pandas strategy)



Training many models in parallel (Cavari strategy)



Batch normalization

32

- normalizing activations in a network

- normalizing inputs to speed up learning

- can we normalize the output of activation functions in hidden layers p
 $(a^{[L]})$

- if we normalize $a^{[L]}$, so the forward propagation on the next layer $(w^{[L+1]}, b^{[L+1]})$ will be faster

- in practice we normalize the output of

$z^{[L]}$, not $a^{[L]}$

Implementing Batch norm:

- Given some intermediate values in NN: $z^{(1)}, \dots, z^{(m)}$

- $\mu = \frac{1}{m} \sum_i z^{(i)}$ compute the ~~mean~~ mean of $z^{(i)}$ in layer L

- $\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$

- $z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}} \rightarrow$ zero deviation

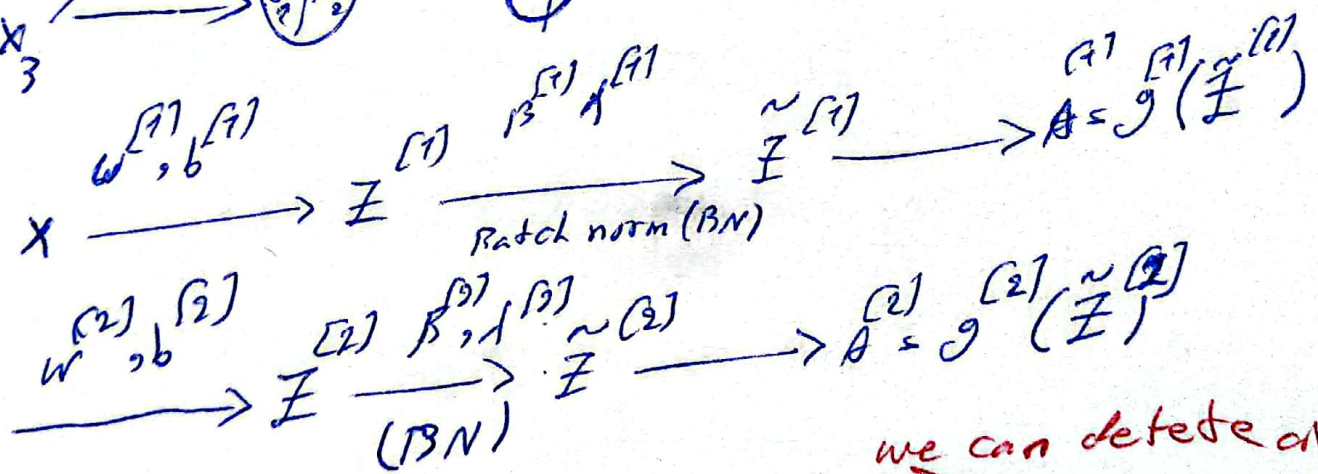
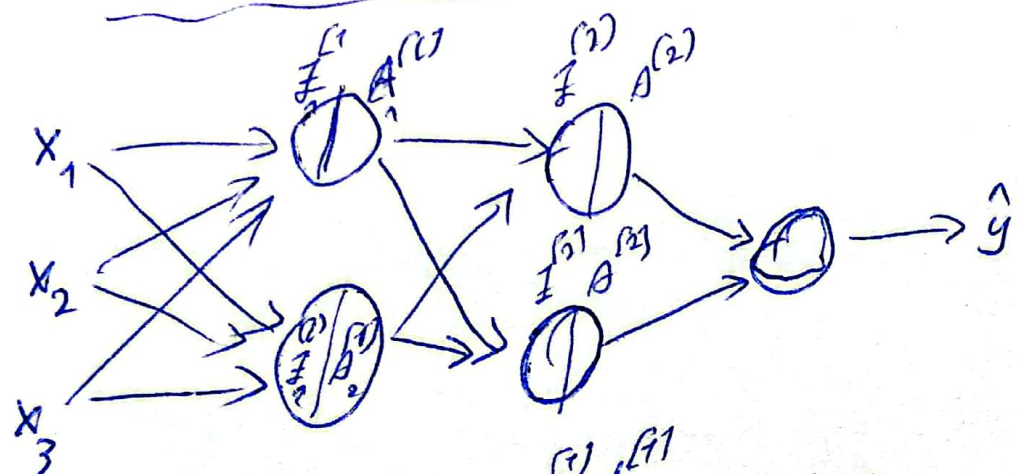
$z^{(i)}$

$\tilde{Z}^{(i)} = \lambda \tilde{Z}_{\text{norm}}^{(i)} + \beta$ learnable parameters of the model

means beside W and b we update λ and β also.

use $\tilde{Z}^{(i)}$ instead of $Z^{(i)}$ to layer computation.

Adding Batch norm to a network



Parameters: $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \dots, W, b$ we can delete all bias terms

$\beta^{(1)}, \lambda^{(1)}, \dots, \beta^{(L)}, \lambda^{(L)}$ because of normalization.

$$W^{(l)} := \alpha \cdot dW^{(l)}$$

$$b^{(l)} := \alpha \cdot db^{(l)} \quad \text{X will be deleted}$$

$$\beta^{(l)} := \alpha \cdot d\beta^{(l)}$$

$$\gamma^{(l)} := \alpha \cdot d\gamma^{(l)}$$

~~the dimension of β and γ is the~~

the dimension of β and γ is the

Same as $b^{(l)}$, means, $(n^{(l)}, 1)$

Implementing Gradient descent using (BN).

for $t: 1, \dots, \text{num-mini-batches}$:

compute forward prop on X^{t+1}

in each hidden layer, use BN to replace $Z^{(l)}$ with $\tilde{Z}^{(l)}$.

use back prop to compute $dW^{(l)}$, $db^{(l)}$, $d\beta^{(l)}$, $d\gamma^{(l)}$

update parameters: $W^{(l)} := W^{(l)} - \alpha \cdot dW^{(l)}$

$$\beta^{(l)} := \beta^{(l)} - \alpha \cdot d\beta^{(l)}$$

$$\gamma^{(l)} := \gamma^{(l)} - \alpha \cdot d\gamma^{(l)}$$

Why batch norm work
does very well?

35

Batch norm at test time

$$\mu = \frac{1}{m} \sum_i Z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (Z^{(i)} - \mu)^2$$

BN formula

$$Z_{\text{norm}}^{(i)} = \frac{Z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{Z}^{(i)} = \gamma \cdot Z_{\text{norm}}^{(i)} + \beta$$

at test time

μ, σ^2 : estimate using exponentially weighted average (across mini-batches)

$x^{(1)}, x^{(2)}, \dots$

$\downarrow$
 $\mu^{(1)}, \mu^{(2)}, \dots$

θ_1

θ_2

$\rightarrow \mu$
 $\rightarrow \sigma^2$

these are computed during train the model and will be used in test time

$$\rightarrow Z_{\text{norm}}^{(i)} = \frac{Z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

are computed during training time.

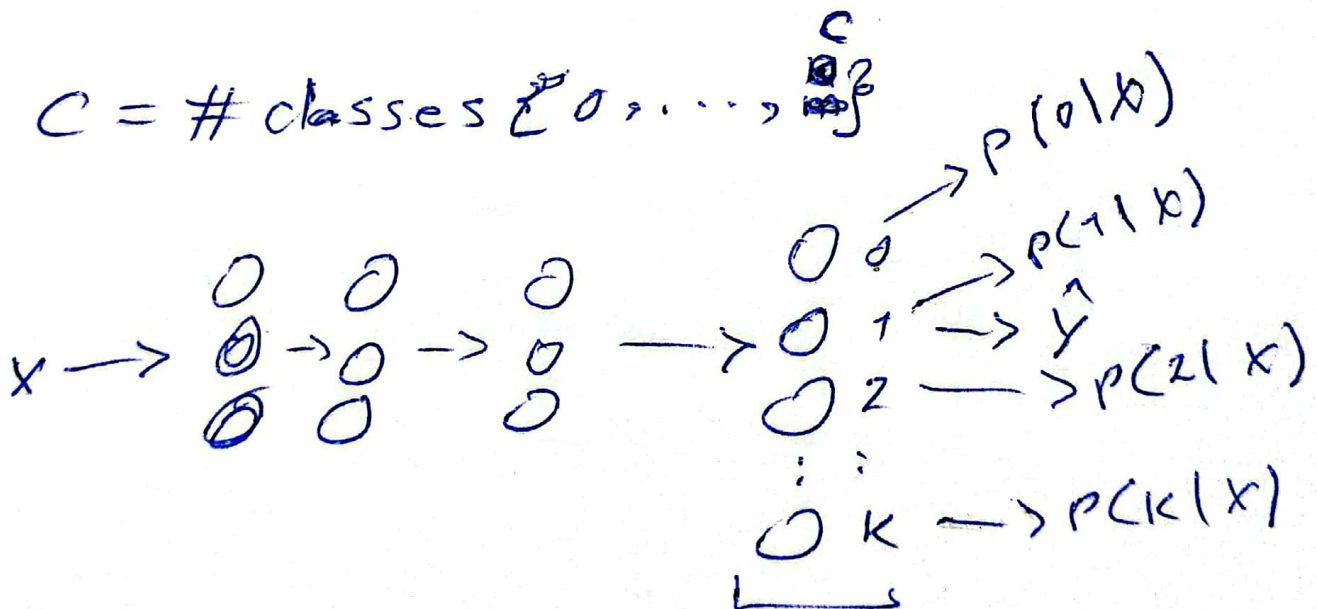
36

$$\hat{I}^{(i)} = y I_{\text{norm}}^{(i)} + \beta$$

Softmax regression

a generalization of softmax regression

$C = \# \text{ classes } \{0, \dots, C-1\}$



$$\hat{y} = (0, 1)$$

number of classes (probabilities)

$$\hat{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Softmax layer (output layer)

37

$$Z^{[L]} = W^{[L]} A^{[L-1]} + b^{[L]}$$

Activation function

(softmax)

$$t = e^{Z^{[L]}}$$

↓

$(a, t) \rightarrow (a, t)$

$$a^{[L]} = \frac{e^{Z^{[L]}}}{\sum_{i=1}^c t_i} \rightarrow \text{number of classes}$$

$$a_i^{[L]} = \frac{t_i}{\sum_{i=1}^c t_i}$$

exp:

$$Z^{[L]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \Rightarrow t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix} = \begin{bmatrix} 148.4 \\ 7.4 \\ 0.4 \\ 20.1 \end{bmatrix}$$

$$\sum_{i=1}^4 f_i = 176.3 \rightarrow \text{sum}(148.4, 7.4, 0.4, 20.1) \quad (38)$$

$$Q = \frac{f}{176.3} =$$

$$\begin{bmatrix} \frac{148.4}{176.3} \\ \frac{7.4}{176.3} \\ \frac{0.4}{176.3} \\ \frac{20.1}{176.3} \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}$$

Sum to 1

class 1 84% chance
 class 2 4% chance
 class 3 0.2% chance
 class 4 11% chance

Sum to 100%

Training a softmax classifier

(39)

exp) $Z = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix}$; $t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix}$

$g^{(L)}(Z^{(L)}) = \begin{bmatrix} e^5 / (e^5 + e^2 + e^{-1} + e^3) \\ e^2 / (e^5 + e^2 + e^{-1} + e^3) \\ e^{-1} / (e^5 + e^2 + e^{-1} + e^3) \\ e^3 / (e^5 + e^2 + e^{-1} + e^3) \end{bmatrix} = \begin{bmatrix} 0.892 \\ 0.092 \\ 0.002 \\ 0.114 \end{bmatrix}$

$\downarrow$ a

Hardmax vs. Softmax

$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.89 \\ 0.092 \\ 0.002 \\ 0.11 \end{bmatrix}$
--	--

- Softmax regression generalizes logistic regression to C classes.

- if $C=2$, softmax reduces to logistic regression.

loss function of softmax

40

$$L(\hat{y} \rightarrow y) = - \sum_{j=1}^C y_j \log(\hat{y}_j) \rightarrow \text{loss function}$$

cost function

$$J(w^{[1]}, b^{[1]}, \dots) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{[i]}, y^{[i]})$$

$$Y = [y^{(1)}, y^{(2)}, \dots, y^{(m)}]$$

$$= \begin{bmatrix} 0 & \vdots & \dots & m \\ 0 & \vdots & \dots & m \\ 1 & \vdots & \dots & n \\ \vdots & \vdots & \dots & n \\ c & \vdots & \dots & c \end{bmatrix}$$

Gradient descent with softmax

Back propagation

$$\frac{dJ}{dZ^{[L]}} = \hat{y} - y$$

(c, 1)

$\rightarrow \frac{dJ}{dZ^{[L]}}$

Deep learning frameworks

Q1

- Caffe/Caffe2
- CNTK
- DL4J
- Keras
- Lasagne
- mxnet
- Paddle Paddle
- Tensorflow
- Theano
- Torch

TensorFlow

```
import numpy as np
import tensorflow as tf

W = tf.Variable(0, dtype=float32)

optimizer = tf.keras.optimizer.Adam(0.1)

cost = W to xW + 25
```

```
def train_step():
```

```
    with tf.GradientTape as tape:
```

$$\text{cost} = w^2 - 10w + 25$$

```
    trainable_variables = [w]
```

```
    grads = tape.gradient(cost, trainable_variables)
```

```
    optimizer.apply_gradients(zip(grads, trainable_variables))
```

~~def~~

```
def train_step():
```

```
    result: 0.092 ...
```

```
exp:
```

```
w = tf.Variable(0, dtype=tf.float32)
```

```
x = np.array([1.0, -10.0, 25.0], dtype=np.float32)
```

```
optimizer = tf.keras.optimizers.Adam(0.1)
```

```
def cost_fn():
```

```
    cost = x[0]*w2 + x[1]*w + x[2]
```

```
    return
```

```
optimizer.minimize(cost_fn, [w])
```

an alternative
for gradient tape